

一、 Pytorch 的核心特性有哪些？

1. 动态计算图：采用动态图机制，代码运行时实时构建计算图，调试更直观便捷；内置 **autograd** 自动微分引擎，可自动完成梯度求解，适配反向传播训练。
2. **Pythonic** 编程风格：语法贴近原生 Python，能无缝衔接 Python 生态的 **numpy**、**matplotlib** 等科学计算库。
3. 强大的 **GPU** 加速：张量可轻松迁移到 **CUDA/GPU** 设备，一行代码就能实现硬件加速，大幅提升训练速度。
4. 灵活的神经网络模块生态：**nn.Module** 模块化设计，支持灵活搭建、复用网络层，能快速构建复杂深度学习模型。
5. 工业级部署能力：支持 **TorchScript**、**ONNX** 转换，可部署到服务器、移动端、边缘设备，打通科研到生产的流程。
6. 分布式训练支持：内置 **DistributedDataParallel** 等工具，便捷实现多卡、多机器分布式训练。

二、 搭建深度学习模型的基本流程有哪些？

搭建深度学习模型，首先需要明确任务目标并开展数据准备工作，采集、清洗原始数据，将数据集划分为训练集、验证集与测试集，完成归一化、数据增强等预处理，再构建批量数据加载流水线；接着基于任务特点搭建网络模型，使用模块化方式定义网络层与前向传播逻辑；随后设置学习率、批次大小、训练轮数等超参数，选定合适的损失函数与优化器；之后开启迭代训练循环，每一轮训练中执行前向传播计算预测结果、求解损

失、梯度清零、反向传播、参数更新一系列操作；每个训练轮次结束后，在验证集上进行无梯度推理，计算相关评估指标，监测模型是否出现过拟合；全部训练完成后，使用独立测试集评估模型泛化能力，最后保存模型权重，完成模型的后续推理或部署工作。

三、 激活函数 softmax 和 sigmoid 函数分别用于哪种分类场景？

- Softmax 函数：单标签分类任务
- Sigmoid 函数：二分类任务、多标签分类任务

四、 卷积神经网络（CNN）和全连接神经网络（FC）的差异有哪些？

全连接网络相邻层所有神经元相互连接，参数量庞大，输入图像时需要展平为一维向量，会丢失空间位置信息，仅适合一维数据；而卷积神经网络依靠局部感受野、权值共享与池化操作，大幅减少模型参数，能够保留图像二维空间结构，自动提取边缘、纹理等局部空间特征，对图像平移、轻微形变具备更好的适应性，更适合图像这类具备局部相关性的数据。二者最大的不同在于特征提取方式，全连接网络依赖人工预处理特征，CNN 可自动学习空间特征，因此卷积神经网络广泛应用于计算机视觉任务，全连接网络多用于表格一维数据场景。

五、 利用 YOLO 官方提供的解决方案，尝试进行 OBB 边界检

测方面的案例复现。

见 `yolo_obb.ipynb`。